Lizenz und fremde Bestandteile

Was schiefging, wie es gefunden wurde, und warum die vorhandenen Proben es nicht sahen

Prof. Dr.-Ing. Ralph Wystup

31. Oktober 2026

Fassung 1.9 — gegenüber 1.8 neu: B23, die Ersetzung lief über ihr eigenes Ergebnis. Fassung 1.8: B22, die fehlende zweite Kurve im Zeitdiagramm. Fassung 1.7: B18 bis B21 aus dem Bau des Galerkin-Lösers für den Browser (Prüfmaß durch null, der Auftrieb als empfindlicher Zeiger, Stabilität über die kennzeichnende Zeit, Einbetten fremder Module). Fassung 1.6: B17, der fehlende Kurzfassungsreiter in der HTML. Fassung 1.5: B11 bis B16 aus dem Bau des Browser-Labors, der Ablage und des unabhängigen zweiten Lösers.

Dieses Blatt führt jeden Fehler, der beim Aufbau des Lösers gefunden wurde. Zu jedem steht nicht nur die Behebung, sondern die Antwort auf die Frage, die wichtiger ist: **Warum hat die vorhandene Prüfung ihn durchgelassen?** Erst diese Antwort führt zu einer Prüfung, die beim nächsten Mal greift.

B1 Péclet-Zahl aus dem Volumenstrom statt aus der Geschwindigkeit

Datei `konvektion.py`, Fassung 1.0 → 1.1

Der Scharfetter-Gummel-Stempel bekam `Pe = u_f·A·d/ν` statt `Pe = u_f·d/ν`. Bei einer Flächenhöhe von 0,0195 m war die Péclet-Zahl damit rund fünfzigfach zu klein. Der Stempel entartete zu reiner Diffusion; der konvektive Anteil fehlte fast vollständig.

Warum die Probe es nicht sah: Die Eigenschaftsprobe prüfte die richtige Beziehung $g_{\text{hin}} - g_{\text{rück}} = u_f A$ — aber mit $A = 1$. Bei $A = 1$ sind Volumenstrom und Geschwindigkeit zahlengleich, und der Fehler ist unsichtbar. Eine Prüfung mit dem Sonderwert, der den Fehler zudeckt, ist keine Prüfung.

Jetzt: Die Probe läuft mit $A = 0{,}0195$ — absichtlich ungleich eins. Dimensionsproben gehören grundsätzlich mit Werten gefahren, bei denen sich die Größen unterscheiden.

B2 Druckquelle ohne Vorzeichen und ohne Zellvolumen

Datei `wirbelstrasse.py`, Fassung 1.0 → 1.1

Die rechte Seite des Drucknetzes stand als `b = div(u*)/dt`. Richtig ist `b = −div(u*)·dx·dy/dt`. `drucknetz_matrix` baut $A = -\Delta \cdot V$; die Leitwerte enthalten die Fläche bereits. Es fehlte also das Zellvolumen $3{,}9 \cdot 10^{-4}$, und das Vorzeichen war verkehrt: Der Druck war 2560-fach zu groß und wirkte in die falsche Richtung. Der Löser flog im ersten Zeitschritt auseinander.

Warum Stufe 2 das nicht sah: In der ausgebildeten Rohrströmung (Hagen-Poiseuille) ist die Divergenz **identisch null**. Die rechte Seite des Drucknetzes ist damit null, gleich welcher Vorfaktor davorsteht. Stufe 2 prüft Zähigkeit und Haftbedingung vollständig — die Druckkopplung prüft sie überhaupt nicht.

Lehre: Eine Prüfung, in der der zu prüfende Term identisch verschwindet, prüft ihn nicht. Jeder Term braucht einen Fall, in dem er von null verschieden ist.

B3 Haftbedingung auf einen inneren Punkt gesetzt

Datei `wirbelstrasse.py`, Fassung 1.1 → 1.2

Die Zeile `us[:, 0] = us[:, -1] = 0` setzte die erste und letzte Zellreihe auf null. Auf dem versetzten Gitter sitzt u aber bei $y = \Delta y/2$ und $y = L_y - \Delta y/2$, also **innen**, nicht an der Wand. Die Haftbedingung steckt bereits im Spiegelwert des Zähigkeitsterms. Zusätzlich stand die Zeile *nach* der Zuweisung des Einströmprofils und löschte damit die Einströmung in der Eckzelle; nach der Projektion wurde sie erneut gesetzt. Der Zufluss änderte sich also **nach** der Druckkorrektur — und genau diese Änderung blieb als Divergenz stehen: gemessen 6,97 in Zelle (0,0).

Warum es vorher nicht auffiel: Es gab keine Probe auf die Divergenz **nach** dem vollständigen Zeitschritt — nur eine auf das Drucknetz allein. Zwischen „die Projektion rechnet richtig" und „am Ende des Zeitschritts ist die Knotenregel erfüllt" liegt alles, was danach noch an den Feldern gedreht wird.

Jetzt: `knotenregel_probe.py` (Fassung 1.0) misst in **jedem** Zeitschritt die Divergenz auf den Feldern, wie der Schritt sie endgültig hinterlässt — nicht auf einem Zwischenstand. Gemessen auf dem Netz 110 × 21 über 384 Schritte: größte Divergenz $3{,}5 \cdot 10^{-13}\,\mathrm{s}^{-1}$, bezogen auf $|u|/\Delta x$ sind das $2{,}8 \cdot 10^{-15}$ — Rundungsschwelle. Dafür hat `wirbelstrasse.lauf` einen Nur-Lesen-Haken bekommen (Fassung 1.4); dass er die Rechnung nicht verändert, wird nicht behauptet, sondern im selben Programm geprüft: derselbe Lauf mit und ohne Haken liefert Bit für Bit dieselbe Verlaufstafel.

B4 Körperkraft ohne den Druckanteil

Datei `wirbelstrasse.py`, Fassung 1.2 → 1.3

Gezählt wurde nur der Impuls, den Konvektion und Zähigkeit in die abgedeckten Zweige tragen. Der Anteil, den die **Druckkorrektur** dort hineinschiebt, fehlte. Beim umströmten Zylinder bei $\mathrm{Re} = 100$ ist gerade der Druckanteil der größere: Ergebnis $c_D = 0{,}60$ statt 3,23, also Faktor 5,4 zu klein.

Warum es nicht auffiel: $c_D = 0{,}60$ ist eine *plausibel aussehende* Zahl — Widerstandsbeiwerte dieser Größenordnung gibt es. Ohne den Vergleich mit einer äußeren Referenz wäre der Fehler nicht zu bemerken gewesen. Genau dafür ist der Benchmark da.

Lehre: Eine Größe, deren Zahlenwert für sich genommen vernünftig aussieht, ist dadurch nicht geprüft. Nur der Abgleich gegen eine Referenz, die man nicht beeinflussen kann, trennt richtig von plausibel.

B5 Ausbleibende Ablösung für Auflösung gehalten — und nachgewiesen

kein Programmfehler, sondern eine offene Frage, die entschieden werden musste

Auf dem Netz mit 10 Zellen über dem Durchmesser blieb c_L bei $-0{,}055$ flach: keine Wirbelablösung. Zwei Erklärungen waren möglich — zu grobes Netz oder zu kurze Zeit. Statt zu raten wurde das feinere Netz gerechnet: Dort stieg c_L auf $\pm 0{,}89$ und schwang in einen stehenden Grenzzyklus ein. **Es war die Auflösung.**

Nebenbefund zur Ablesung: Die Zwischenmeldungen des feinen Laufs schienen eine Schwingung mit rund 5 s Periode zu zeigen. Das ist ein Trugbild: Ausgegeben wurde alle 0,3333 s, die wahre Ablöseperiode beträgt 0,356 s. Die Schwebung zwischen beiden ergibt

rechnerisch $|1/0{,}3333 - 1/0{,}356|^{-1} = 5{,}3$ s — genau das Beobachtete. Die Auswertung der Strouhalzahl läuft deshalb über die **vollständige** Zeitreihe, nie über die Meldezeilen.

Lehre: Eine Zwischenmeldung ist eine Stichprobe. Wer aus Stichproben eine Periode abliest, muss ihr Abtastintervall gegen die erwartete Periode halten.

B6 Geisterwerte als Konstante statt als Matrixeintrag

Datei `impuls_matrix.py`, Fassung 1.0 → 1.1

Beim ersten Aufbau der Impulsmatrix wanderten **alle** Randbeiträge in den Vektor b — auch die, die Kopien (freier Ausstrom) oder Spiegelungen (Haftbedingung) innerer Unbekannter sind. Die Sprachprobe bestand trotzdem, weil sie $Mu + b$ am selben Zustand auswertet, aus dem b abgelesen wurde.

Warum die Probe es nicht sah: Sie prüfte die *Wirkung* am aktuellen Zustand, nicht die *Matrix*. Für eine reine Residuenprobe reicht das; für eine Matrix, die später implizit gelöst oder als Jacobi-Matrix gebraucht wird, nicht.

Jetzt: Geisterwerte, die von inneren Unbekannten abhängen, stehen als Matrixeintrag auf dem Knoten, den sie kopieren beziehungsweise spiegeln (Faktor -1). Nur das vorgegebene Einströmprofil ist echte Konstante. Zusätzlich geprüft wird, dass die Matrix für $u \to 0$ **symmetrisch** wird — das passive Widerstandsnetz des Feld-Labors.

B7 Dateiliste läuft in die Argumentgrenze der Brücke

Datei `/workspace/sichere_nach_c.py`, Fassung 1.0 → 1.1 — **ein Werkzeugfehler, kein Projektfehler**, aber er hätte dieses Projekt getroffen.

Die zu schiebenden Dateinamen wurden zu **einem** Brückenargument zusammengesetzt. Die Brücke lässt je Argument 160 Zeichen durch; alles darüber wird abgeschnitten. Für dieses Projekt sind es 14 Dateien, zusammen rund 380 Zeichen — mehr als die Hälfte wäre verschwunden, **ohne Meldung**.

Warum es nie auffiel: Die bisherigen Zielordner hatten zwei bis neun Dateien mit kurzen Namen; die Liste blieb unter der Grenze. Ein Fehler, der erst ab einer gewissen Größe auftritt, wird von kleinen Fällen nicht gefunden — und alle bisherigen Fälle waren klein.

Jetzt: Die Liste wird in Häppchen zerlegt, die sicher hindurchpassen. Dazu ein zweiter Mangel an derselben Stelle: Nach dem Schieben wurde gemeldet, was die Brücke *sagte*, nicht was drüben *liegt*. Jetzt wird die Dateiliste erneut gelesen und jede Datei mit ihrer Größe bestätigt.

B8 `import math` innerhalb einer Funktion

Datei `schaubild.py`

Dasselbe Muster wie am 30.10.2026 in `zonen_aus_feld.py` des REM-Projekts: Ein Modul wird nur innerhalb einer Funktion eingebunden. Solange genau diese Funktion läuft, geht es gut; wird der Aufruf später an eine andere Stelle gezogen, bricht es ab.

Warum es trotz des bekannten Befunds wieder passierte: Der Import entstand beim nachträglichen Einfügen einer Filterzeile, und an dieser Stelle war das Modul noch nicht oben eingebunden. Es gab keine Prüfung, die danach sucht.

Jetzt: behoben; und bei jeder Durchsicht wird `grep -n "import " ` gegen den Funktionsrumpf gehalten.

B9 Das Methodenblatt beschrieb ein anderes Programm

Datei `METHODE_UND_NACHWEIS.md`, §5.3

Der Abschnitt beschrieb Stufe 2 als **Rohrströmung** mit $u(r) = 2\bar{u}(1 - r^2/R^2)$ und $\Delta p = 8\mu LQ/(\pi R^4)$. Gerechnet wird aber ein **ebener Kanal** mit $u(y) = G/(2\nu)\, y(h - y)$. Schwerer wiegt der zweite Satz: Es stand dort, diese Stufe prüfe „die Zähigkeitswiderstände und die Druckkopplung". Die Druckkopplung prüft sie **nachweislich nicht** — genau deshalb konnte Befund B2 entstehen.

Warum es nicht auffiel: Der Abschnitt wurde geschrieben, bevor das Programm stand, und danach nicht gegen das Programm gehalten. Eine Beschreibung, die der Rechnung vorausgeht, muss nach der Rechnung erneut gelesen werden.

Jetzt: §5.3 steht mit der richtigen Formel, der gemessenen Tafel und einem eigenen Absatz „Was diese Stufe NICHT prüft". Die noch nicht gerechnete Taylor-Aris-Dispersion ist als Vorhaben gekennzeichnet, nicht als Nachweis.

B10 Die Wirbelstärke im Bild war um eine Zelle versetzt

Datei `bild.py`, Fassung 1.0 → 1.1 — **der Fehler saß im Beweisstück selbst.**

Die Wirbelstärke wurde gebildet als

```
dvdx = (v[1:,:] - v[:-1,:]) / dx      Form (nx-1, ny+1)
dudy = (u[:,1:] - u[:,:-1]) / dy      Form (nx+1, ny-1)
omega = dvdx[:n,:m] - dudy[:n,:m]
```

also durch Zurechtschneiden auf die kleinere Form. Auf dem versetzten Gitter sitzt `dvdx[k,m]` aber auf der Ecke $((k{+}1)\Delta x,\ m\,\Delta y)$ und `dudy[k,m]` auf $(k\,\Delta x,\ (m{+}1)\Delta y)$. Voneinander abgezogen wurden Werte von **verschiedenen Orten**, je eine ganze Zelle versetzt. Dazu wurde das Ergebnis über die volle Kanalbreite gezeichnet, obwohl es gar nicht bis an den Rand reicht.

Warum es beim Ansehen nicht auffiel: Der Fehler ist von der Ordnung h. Er zerstört das Bild nicht, er verwischt es — und ein verwaschener Wirbel sieht aus wie ein Wirbel.

Warum eine naheliegende Probe ihn auch nicht gefunden hätte: Die ersten drei Prüffelder, die ich dafür geschrieben hatte, waren **linear** (Starrkörperdrehung, Scherung, Dehnströmung). Bei linearen Feldern ist die Ableitung überall gleich; ein um eine Zelle verschobener Ablesepunkt liefert denselben Wert. Die alte Fassung bestand diese drei Proben genauso gut wie die neue — mit Abweichung null. Erst Felder mit **ortsabhängiger** Wirbelstärke decken den Versatz auf:

Prüffeld	ω soll	Fassung 1.0	Fassung 1.1
$v = xy,\ u = 0$	y	0,02158 $= \Delta y$	$2{,}4 \cdot 10^{-15}$
$u = xy,\ v = 0$	$-x$	0,05946 $= \Delta x$	$7{,}6 \cdot 10^{-15}$

Die Abweichung der alten Fassung ist **exakt ein Zellabstand** — der Versatz, Zahl für Zahl bestätigt.

Jetzt: ω wird auf der inneren Gitterecke gebildet, wo beide Anteile denselben Ort haben, und mit der zugehörigen Ausdehnung gezeichnet. Fünf Prüffelder, davon zwei mit ortsabhängigem ω, laufen als Stufe S09 im Prüfprotokoll mit.

Lehre: Ein Prüffeld muss die Eigenschaft *variieren*, die geprüft werden soll. Eine Probe auf Ortsrichtigkeit mit ortsunabhängigen Feldern ist keine Probe. Das ist dieselbe Falle wie in Befund B1, wo $A = 1$ den Unterschied zwischen Geschwindigkeit und Volumenstrom zudeckte.

B11 Die Kernprobe deckte nur eine von zwei Einstellungen ab

Datei `labor/kern_cfd.js`, Fassung 1.0 → 1.1

Der Rechenkern des Browser-Labors ist eine zweite Umsetzung derselben Verfahren und deshalb gegen die Python-Module geprüft — 41 von 41 Posten, alles auf 10^{-16}. Trotzdem fehlte dem Drucknetz ein Stück: Im reinen Neumann-Fall (ohne Ausstromrand) hält die Python-Fassung **einen** Knoten fest, sonst ist das Netz stromlos aufgehängt und die Matrix singulär. Die JavaScript-Fassung tat das nicht.

Warum die Probe es nicht sah: Sie prüfte ausschließlich den Fall **mit** Ausstromrand — den, der im Löser vorkommt. Die Seite bietet aber einen Knopf „nacktes Netz" an, und damit ist der andere Fall Teil der Lieferung. Geprüft war nur die Hälfte dessen, was bedienbar ist.

Gefunden hat es die Knopfprobe, nicht die Kernprobe: Im Browser lieferten „mit Störkörper" und „nacktes Netz" dieselbe Zahl — was bei verschiedenen Netzen nicht sein kann.

Jetzt: Der Bezugsknoten ist in `kern_cfd.js` nachgezogen, und `vergleichsdaten.py` führt den Fall ohne Ausstrom mit — 45 von 45 Posten. Dazu eine Feinheit, die jetzt ausdrücklich dasteht: Weg B kennt keine Einheitszeile, er bilanziert nur. Beide Wege unterscheiden sich daher **genau an diesem einen Knoten**, und dort wird nicht verglichen — so hält es die Sprachprobe in `ns_loeser.py` seit jeher, und die Seite sagt es jetzt in der Überschrift der Zeile mit.

Lehre: Der Prüfumfang richtet sich nach dem, was **bedienbar** ist, nicht nach dem, was im Hauptweg vorkommt. Jeder Knopf, den die Seite anbietet, ist ein Fall, der geprüft sein muss.

B12 Das Surrogat verfehlte sein Kriterium — zweimal, aus zwei Gründen

Dateien `labor/surrogat.py` 1.0 → 1.1, `labor/harmonisch.py` 1.0

Der Zeitverlauf im Browser wird von einem angelernten Ersatzmodell getragen: POD für die Raumformen, dazu ein Zeitgesetz. Das Kriterium war von vornherein gesetzt — weniger als 5 % der Schwankungsbreite Fehler gegen den echten Löser. Es wurde zweimal verfehlt, und beide Male steckte etwas anderes dahinter.

Erster Anlauf: 15,9 %. Angelernt auf Daten bis $t = 8$ s. Die Periode kam 1,3 % zu lang heraus, und über 3,5 Perioden lief daraus ein Phasenfehler auf: 15,8 % nach einer halben Ablöseperiode, wieder 0,5 % nach einer ganzen. Ein Fehler, der **schwingt statt zu wachsen**, ist die Unterschrift eines Phasenfehlers — nicht eines falschen Modells.

Dazwischen ein Fehler in meiner eigenen Diagnose. Um zu prüfen, ob der Grenzzyklus eingeschwungen ist, verglich ich die Schwankungsbreite der **ersten** mit der der **letzten** Momentaufnahme. Die liegen aber an verschiedenen Stellen der Schwingung — der Vergleich maß die Phase, nicht die Drift. Er meldete 7 bis 10 % Abfall und schickte mich auf die falsche Fährte „noch nicht eingeschwungen". Je *ganzer Periode* gemessen, wie es sich gehört, beträgt die Drift 0,003 % über zehn Perioden: Die Daten waren längst eingeschwungen.

Zweiter Anlauf: 7,3 %. Auf eingeschwungenen Daten bis $t = 12$ s traf die DMD die Grundfrequenz 2,524 Hz mit ihren ganzzahligen Oberwellen, die Eigenwerte lagen auf dem Einheitskreis — die Struktur stimmte. Übrig blieb ein Restfehler von 0,17 % in der Frequenz, über 6,3 Perioden 3,84 Grad Phase. Weil das erste Modenpaar 96 % der Schwankung trägt, sind das 7 % im Feld.

Behoben durch die bekannte Struktur, nicht durch einen längeren Lauf. Ein Grenzzyklus ist periodisch; seine Beiwerte sind $a(t) = a_0 + \sum_k [c_k \cos(2\pi k f t) + s_k \sin(2\pi k f t)]$ mit **einer** Grundfrequenz. Das ist keine Annahme, die den Daten übergestülpt wird: Die paarweisen Singulärwerte und die ganzzahligen DMD-Frequenzen zeigen es. Angepasst werden die Beiwerte über das ganze Fenster, f durch eine eindimensionale Suche. Das Modell ist damit **exakt periodisch** — es kann sich keine Phase davonstehlen.

Zeitgesetz	Fehler
angepasster linearer Operator (DMD)	7,33 %
harmonische Anpassung, 8 Oberwellen	**0,187 %**

Faktor 39. Beide Zahlen stehen im Prüfbericht und auf der Seite; die schlechtere wird nicht weggelassen.

Lehre: Ein Fehler, der mit der Grundperiode schwingt und nicht wächst, ist ein Phasenfehler — dagegen hilft Struktur, nicht mehr Daten. Und: Eine Diagnose, die zwei Zeitpunkte vergleicht, muss sie an derselben Stelle der Schwingung nehmen.

B13 Das Prüfprogramm schlug falschen Alarm

Datei `/workspace/pruefe_ablage_c.py`, Fassung 1.1 → 1.2 — **ein Werkzeugfehler.**

Der Byteabgleich gegen C: meldete „16 von 17 Dateien byteidentisch". Beim Wiederholen derselben Prüfung, ohne dass irgendetwas geändert worden wäre: „17 von 17". Auf C: lag also alles richtig — das Prüfprogramm hatte sich geirrt.

Die Ursache ist ein Wettlauf. Die Brücke meldet „gespeichert", sobald sie die Datei abgeschickt hat; der Empfänger im Arbeitsbereich schreibt sie aber erst danach. Wer sofort die Prüfsumme bildet, bekommt ein Bruchstück und sieht einen Unterschied, den es nicht gibt.

Warum das schlimmer ist als gar keine Prüfung: Ein Prüfprogramm, das einmal so und einmal anders antwortet, schickt die Suche in die falsche Richtung — hier hätte es mich dazu gebracht, eine Datei auf C: für beschädigt zu halten und sie noch einmal zu schieben. Dasselbe Programm hatte schon einmal falsch gemeldet (Fassung 1.0: „0 von 12" für Dateien, die alle da waren, weil es am falschen Ort und unter dem falschen Namen suchte). Zweimal derselbe Fehlertyp: **Das Prüfmittel war ungeprüft.**

Jetzt: Die Antwort der Brücke nennt die erwartete Byte-Zahl. Das Programm wartet, bis die Datei diese Größe erreicht hat (höchstens 10 s), und meldet ausdrücklich „unvollständig zurückgekommen", wenn sie es nicht tut — statt dies als Unterschied im Inhalt auszugeben.

Lehre: Auch das Prüfmittel braucht eine Probe. Ein sinnvoller Test dafür ist die Wiederholung: Zweimal dasselbe geprüft muss zweimal dasselbe ergeben. Wo das nicht zutrifft, liegt der Fehler zuerst im Prüfmittel.

B14 Die Größenprüfung hat eine veraltete Datei durchgelassen

Datei `/workspace/sichere_nach_c.py`, Fassung 1.2 → 1.3 — **ein Werkzeugfehler, diesmal mit Wirkung.**

`sichere_nach_c.py` entschied bisher an der **Dateigröße**, was nach C: geschoben werden muss. In `kern_cfd.js` wurde „Fassung 1.0" durch „Fassung 1.1" ersetzt — **gleich viele Zeichen**. Die Datei auf C: galt damit als aktuell und blieb veraltet. Dasselbe traf die gebaute Seite, in der dieselbe Zeichenfolge steckt.

Das war kein hypothetischer Fall: Der Bytevergleich meldete unmittelbar danach „7 von 8 Dateien byteidentisch" und benannte `kern_cfd.js`.

Warum es nie auffiel: Textänderungen ändern fast immer die Länge. Eine Fassungsnummer von 1.0 auf 1.1 zu heben ist der seltene Fall, der es nicht tut — und ausgerechnet der trifft eine Datei, deren Fassung im Kopf steht.

Jetzt: Das Programm führt ein **Verzeichnis** dessen, was es tatsächlich geschoben hat (`SICHERUNG/nach_c_gesandt.json`, Name und Prüfsumme). Geschoben wird, wenn die Prüfsumme von der verzeichneten abweicht **oder** die Größe drüben nicht stimmt. Das Verzeichnis ersetzt den Bytevergleich nicht — es sorgt nur dafür, dass er nichts mehr zu finden bekommt.

Zweiter Teil desselben Befunds: Die Fassungsnummer des Rechenkerns stand an **zwei** Stellen — im Kopf von `kern_cfd.js` und fest eingetragen im Bauskript. Der Kopf wurde auf 1.1 gehoben, das Bauskript blieb auf 1.0, und die Seite meldete die falsche Fassung. `baue_labor.py` liest sie jetzt aus dem Kopf des Kerns. Eine Zahl an zwei Stellen ist eine Zahl zu viel — das ist der Grund für die stehende Regel zu Fassungsnummern, und hier ist der Beleg.

Lehre: Wer an der Größe entscheidet, entscheidet an einem Merkmal, das sich ändern *kann*, aber nicht *muss*. Zur Entscheidung taugt nur die Prüfsumme — und zum Nachweis ohnehin nur der Bytevergleich.

B15 Am Ausstrom fiel die Haftbedingung in den Ecken weg

Datei `fem_galerkin.py`, Fassung 1.0 → 1.1

Der Galerkin-Löser bekam die Randbedingungen über

```
fest = (ein | wand | zyl) & ~aus
```

Der Gedanke dahinter war richtig: Am Ausstrom soll nichts vorgeschrieben werden, dort gilt die natürliche Bedingung. Nur trifft `& ~aus` auch die **beiden Eckknoten**, an denen der Ausstrom auf die Kanalwand stößt — und dort fiel damit die Haftbedingung weg. Richtig ist: Der Ausstrom gibt nur frei, was **ausschließlich** am Ausstrom liegt; Wand bleibt Wand bis zum letzten Knoten.

Was es kostete und warum es trotzdem gefunden wurde. Die Hagen-Poiseuille-Probe lag 3,3 % daneben. Das ist deshalb entlarvend, weil sie auf **Maschinengenauigkeit** aufgehen muss: P2 enthält die Parabel exakt, P1 den linearen Druck exakt — die Lösung liegt also im Ansatzraum und muss bis auf Rundung herauskommen. Eine Probe, deren Sollwert nicht „klein" ist, sondern „null", lässt zwei Knoten von viertausend nicht durchgehen. Nach der Behebung: $4{,}9 \cdot 10^{-14}$ im Profil, Druckgefälle auf zehn Stellen.

Die Fehlersuche ging zuerst in die falsche Richtung. Ich hielt die Dirichlet-Elimination für die Ursache und schrieb sie neu — von umständlichen Listenoperationen auf vier Zeilen Matrixalgebra. Das Ergebnis war **Zahl für Zahl dasselbe**: Die alte Fassung war richtig, nur hässlich. Erst danach habe ich die Randmarkierung selbst angesehen. Dass die neue Fassung dieselben Zahlen lieferte, war dabei der nützliche Befund — er schloss die Elimination als Ursache aus.

Lehre: Eine Probe, deren richtiges Ergebnis exakt null ist, ist mehr wert als zehn, die nur „klein" verlangen. Und wenn eine Umschreibung dieselben Zahlen liefert, hat man nicht den Fehler behoben, sondern eine Ursache ausgeschlossen — das ist auch etwas wert, aber es ist etwas anderes.

B16 Ein Vergleich, der die Auflösung zeigte statt des Verfahrens

Datei `labor/baue_labor.py`, Fassung 1.1 → 1.2

Für den Reiter „Netzwerk gegen Galerkin" wollte ich beide Rechnungen nebeneinander abspielen. Der erste Versuch stellte die gespeicherten Bilder des **groben** Netzwerklaufs (220 × 41) neben die des Galerkin-Laufs. Das Ergebnis sah nach einem großen Unterschied aus — und war keiner:

	größtes $\lvert\omega\rvert$
Netzwerk, 220 × 41	116
Galerkin, 81 Knoten am Zylinder	268

Faktor 2,3. Das ist nicht der Unterschied der **Verfahren**, sondern der der **Auflösung**: Das grobe Netz kann die Wandgradienten gar nicht darstellen. Ein Bild, das so entsteht, beweist nichts und legt eine falsche Fährte — es sähe aus, als wichen die Verfahren stark voneinander ab.

Jetzt: Der Vergleich nimmt das Netzwerkfeld vom **feinsten** Netz (880 × 164) und mittelt es blockweise auf dasselbe Anzeigegitter wie das Galerkin-Feld. Gleiche Auflösung, gleiche Darstellung — erst dann zeigt die Differenz das Verfahren.

Zweiter Teil desselben Befunds: die Phasenzuordnung. Beide Rechnungen laufen nicht im Gleichtakt; zugeordnet werden muss über den Zustand des Grenzzyklus. Zuerst nahm ich dafür $(c_L,\ \dot{c}_L)$, beides auf das eigene Maximum bezogen. Das rohe c_L ist aber von Schritt zu Schritt verrauscht; **seine Ableitung wird vom Rauschen beherrscht**, und die Normierung macht das Rauschen zum Maßstab. Gemessen: mittlerer Zuordnungsabstand 0,31 statt der erwarteten 0,01, siebzig Prozent der Bilder schlecht zugeordnet. Jetzt kommt die Phase aus einer **harmonischen Anpassung** an c_L — beide Signale sind Grenzzyklen mit bekannter Grundfrequenz, die Anpassung ist glatt und eindeutig.

Dritter Teil, derselbe Fehlertyp noch einmal. Mit dem feinen Netz war der Vergleich immer noch schief: Das Netzwerkfeld wurde über 8 × 8-**Blöcke gemittelt** auf das Anzeigegitter gebracht, das Galerkin-Feld dagegen am Punkt **interpoliert**. Die Mittelung glättet genau die Wandspitze weg, die der Vergleich zeigen soll — die Spitzenwirbelstärke kam auf 127,5 gegen 268 heraus, also wieder Faktor 2,1, diesmal allein durch die Auswertung. Zur Einordnung: Die Grenzschicht ist hier rund $D/\sqrt{\mathrm{Re}} = 10$ mm dick, der Mittelungsblock 20 mm — es wurde über die ganze Grenzschicht hinweggemittelt. Jetzt werden **beide** Felder linear an denselben Anzeigepunkten ausgewertet.

Lehre: Zwei Dinge nebeneinanderzulegen ist erst dann ein Vergleich, wenn **alles** andere gleich ist — Auflösung, Phase **und die Auswertung selbst**. Dreimal in Folge habe ich an dieser einen Stelle etwas übersehen, und jedes Mal sah das Ergebnis nach einem großen Unterschied der Verfahren aus. Und: Eine Ableitung eines verrauschten Signals ist kein Messwert, sondern eine Rauschverstärkung; wo eine Phase gebraucht wird, nimmt man die angepasste Schwingung.

B17 Die Kurzfassung war in jedem Dokument — nur nicht in der HTML, wo sie bestellt war

Sein Auftrag vom 31.10.2026 war wörtlich: *„ich breauche noch eine kurzfassung, 1 seite in dr html, Aufgabe, Zielsetzung, methode und fazit."* Gebaut wurde `KURZFASSUNG.md`, daraus `Kurzfassung_1.0.pdf`, zwei Seiten, mit genau diesen vier Abschnitten — und später auf seinen Zusatzwunsch erweitert um die Nachbildung der Navier-Stokes-Gleichungen im Netzwerk und den Zweck des Surrogats. Nur im Labor selbst stand sie **nicht**: `CFD_Labor_1.1.html` hatte sechs Reiter, und keiner davon war die Kurzfassung.

Gefunden hat es seine Frage „ist das die aktuelle version?" — beim Nachsehen, ob die Datei auf C: der neueste Stand ist, habe ich die Reiterleiste gelesen.

Warum die vorhandene Prüfung es nicht sah. Die Knopfprobe `pruefe_labor.js` geht `window.PRUEFSTAND.reiter` durch und prüft jeden Reiter darauf, dass er sich öffnet und Inhalt zeigt. Sie prüft also, ob die **vorhandenen** Reiter funktionieren — nie, ob ein **geforderter** fehlt. Eine Liste, die aus der Seite selbst stammt, kann eine Lücke in der Seite nicht melden: Sie ist gegenüber dem Soll blind. 36 von 36 Punkten waren grün, und alle 36 sagten nichts über diesen Fall.

Dazu der zweite Grund, der eigentliche: **Der Prüfplan hatte kein Kriterium dafür.** Grundsatz 40 verlangt, dass jede neue Aussage des Auftraggebers als Kriterium in den Prüfplan wandert. Dieser Satz wurde ausgeführt, aber nicht eingetragen — K1 bis K12 decken Verfahren, Nachweis und Ablage ab, keines die Oberfläche. Was nicht als Kriterium dasteht, wird beim Durchgehen vor der Abgabe nicht gefragt.

Behoben. Reiter „Kurzfassung" in `vorlage.html`, `@@KURZFASSUNG@@` aus `KURZFASSUNG.md` in `baue_labor.py`, Labor **1.2**. Er steht vor „Verfahren & Nachweis", also am Anfang der Textreiter; Einstieg bleibt „Vergleichsverfahren", weil der erste Bildschirm bedienbar sein soll und nicht Text (Grundsatz „Werkzeug vor Text").

Prüfung, die es künftig fängt. `pruefe_labor.js` prüft die Reiterliste jetzt gegen eine **fest eingetragene Sollliste** aus den Aussagen des Auftraggebers, nicht nur gegen sich selbst — ein fehlender Reiter fällt durch, ein unerwarteter ebenso. Dazu K13 im Prüfplan.

Lehre: Eine Prüfung, die ihre eigene Liste aus dem Prüfling bezieht, kann nur Defekte finden, keine Lücken. Für jede Soll-Aussage braucht es eine Liste, die **außerhalb** des Prüflings steht.

B18 Ein Prüfmaß, das durch eine Null teilt

Beim Aufbau des Galerkin-Kerns für den Browser verglich ich die Matrizen mit Python über Summe und Quadratsumme, jeweils relativ. Die Quadratsummen stimmten auf 1e-14, die **Summen** fielen mit 0,56 und 2,2 durch. Beides an derselben Matrix.

Der Kern war richtig. Die Gesamtsumme von **K**, **Bx** und **By** ist *von Natur aus null* — genau das besagt der Satz „die Steifigkeit vernichtet das konstante Feld", der zwei Zeilen darüber mit 3,8e-15 bestand. Der relative Vergleich teilte also durch Rundungsrauschen.

Warum die vorhandene Prüfung es nicht sah. Sie *war* die Prüfung — sie hat Alarm geschlagen, nur an der falschen Stelle. Das ist der gefährlichere Fall: Ein Prüfmaß, das falschen Alarm gibt, kostet Zeit und gewöhnt einen daran, Alarme wegzuerklären.

Behoben. Gemessen wird jetzt am Maßstab der Matrix selbst, der Wurzel aus der Quadratsumme — die ist nie null.

Lehre: Vor jedem relativen Vergleich ist zu fragen, ob der Nenner null sein *kann*. Wo eine Größe aus Gründen verschwindet, braucht sie einen eigenen Maßstab.

B19 Der Auftrieb fällt zuerst — er ist der empfindlichere Zeiger

Der Blockvorkonditionierer des Sattelsystems hatte den Zeitschritt im Zähler statt im Nenner. Richtig ist $\hat{S}^{-1} \approx \nu M_p^{-1} + K_p^{-1}/\Delta t$; bei $\Delta t = 2 \cdot 10^{-3}$ lag der Druckanteil damit um $\Delta t^2 = 4 \cdot 10^{-6}$ daneben. MINRES lief in jedem Zeitschritt in die Schrittgrenze und brach **unfertig** ab.

Das Aufschlussreiche ist, was man dabei sah:

Größe	Abweichung gegen Python
Widerstandsbeiwert c_D	6e-6
Auftriebsbeiwert c_L	**63 %**

Der Widerstand sah fast richtig aus. Er ist eine Summe gleichgerichteter Beiträge und verträgt einen unfertigen Löser. Der Auftrieb ist die **kleine Differenz zweier großer Zahlen** — die Druckkräfte auf der oberen und der unteren Zylinderhälfte heben sich fast auf. Was übrig bleibt, ist klein gegen das, woraus es entsteht, und zerfällt als erstes.

Warum die vorhandene Prüfung es nicht sah. Sie sah es — aber erst, weil c_L mitgeprüft wurde. Hätte ich nur c_D geprüft, wäre der Fehler mit 6e-6 durchgegangen.

Lehre: In jedem System gibt es Größen, die aus Differenzen entstehen. Sie sind die empfindlichen Zeiger und gehören in die Prüfung, auch wenn eine andere Größe „wichtiger" scheint. Ein Löser, der die unempfindliche Größe trifft, ist nicht geprüft.

B20 Eine Prüfung, die vor der kennzeichnenden Zeit aufhört

Der Galerkin-Kern im Browser bestand 45 von 45 Prüfungen, darunter 40 Zeitschritte gegen Python auf 1,6e-9. Im Dauerlauf platzte **dieselbe Rechnung** bei t = 0,372 s mit NaN.

Ursache gemessen, nicht vermutet: die Courant-Grenze der expliziten Konvektion.

Zeitschritt	CFL	Ergebnis
2,0e-3	0,308	NaN bei Schritt 186 (t = 0,372 s)
1,0e-3	0,154	bis t = 1,0 s stabil, c_D 2,8494
5,0e-4	0,077	stabil, c_D 2,8492
2,5e-4	0,038	stabil, c_D 2,8491

Die 40 Prüfschritte entsprachen t = 0,08 s — ein Fünftel des Weges bis zum Platzen.

Warum die vorhandene Prüfung es nicht sah. Sie prüfte *Übereinstimmung*, nicht *Bestand*. Beide Rechnungen waren in den ersten 40 Schritten gleich — und blieben es auch, bis die eine abbrach. Übereinstimmung über eine kurze Strecke sagt über Stabilität nichts; Stabilität ist eine Aussage über die *Dauer*.

Behoben. Der Kern bestimmt den Zeitschritt jetzt selbst aus der Courant-Bedingung des Netzes ($\Delta t \le C\,(h_{\min}/2)/u_{\max}$, C = 0,15, also ein Sicherheitsabstand von zwei gegen das Gemessene) und meldet, wenn eine Vorgabe gekappt wurde. Die Prüfung läuft jetzt bis t = 1,2 s, über die Zeit hinaus, in der die Ablösung anläuft — 1280 Schritte.

Lehre: Bei jedem Zeitschrittverfahren muss die Prüfung mindestens so lange laufen wie die **kennzeichnende Zeit des Vorgangs**. Was davor grün ist, ist nicht geprüft, sondern nur noch nicht gescheitert.

B21 Ein Parse-Fehler legt den ganzen Block still — und die Seite lädt „fehlerfrei"

Triangle (Shewchuk) sollte als WebAssembly in die eine HTML-Datei. Drei Anläufe, drei verschiedene Fehler, alle derselben Art: Zwei Dateien, die als getrennte Module geschrieben sind, kollidieren, wenn man sie aneinanderhängt.

1. Die Hülle enthielt noch ihr `require` — `createModule` stand zweimal da.

2. Danach stand `Module` zweimal da: einmal `var` aus dem Emscripten-Erzeugnis, einmal `let` aus der Hülle.
3. Danach lief es durch, wies aber nichts zu: Das Erzeugnis setzt seine Fabrik nur, wenn **beide** da sind, `module` *und* `exports`. Ich hatte nur `module` gesetzt.

Das Tückische sind die ersten beiden. Ein doppelt deklarierter Name ist ein **Parse**- Fehler: Der ganze Skriptblock läuft dann gar nicht erst an. Die Seite sah trotzdem normal aus, alle Reiter gingen auf, und die Diagnoseschnittstelle meldete brav `KERN_FEM: object` — denn das stand in einem *anderen* Block.

Warum die vorhandene Prüfung es nicht sah. Die Knopfprobe prüft, ob Reiter aufgehen und Knöpfe rechnen. Ein Stück, das noch an keinem Knopf hängt, kommt darin nicht vor. Gefunden hat es erst die Probe, die gezielt nach `window.TRIANGLE` fragte und ein Netz erzeugen ließ.

Lehre: Beim Einbetten fremder Module ist der Geltungsbereich die ganze Arbeit — jedes Stück in eine eigene Funktion, nach außen nur ein Name. Und: Nach jedem Einbetten muss geprüft werden, dass das Eingebettete **etwas tut**, nicht nur dass es dasteht.

B22 Eine Kurve fehlte, und die Probe zählte Farben

Im Reiter „Galerkin" steht ein Zeitdiagramm mit beiden Wegen übereinander; die Legende nennt Galerkin (blau) und Netzwerk (rot). Gezeichnet war nur **eine** Kurve.

Ursache: Der Netzwerkweg führt seine Bahn am *Lauf*, nicht am *Zustand* — so ist es im Reiter „Ganzer Löser" gebaut, und ich habe `ZN.bahn` abgefragt, was es dort nicht gibt. Der Zeichner bekam `undefined` und übersprang die Reihe stillschweigend.

Warum die vorhandene Prüfung es nicht sah. Der Prüfposten lautete *„die Beiwerte beider Wege laufen über der Zeit"* und maß die **Zahl der Farbtöne** im Bild: 652, also bestanden. Eine Zahl von Farbtönen sagt aber nicht, *welche* Farben da sind — eine einzelne Kurve mit Achsen und Gitterlinien liefert längst genug davon. Das Kriterium hieß „beide Wege" und maß „irgendetwas Buntes".

Gefunden hat es das **Ansehen des Bildes**, nicht die Probe (Hausregel „Bild ansehen, nicht nur Prüfzahl"). Dabei fiel zugleich auf, dass der Anfahrstoß — c_D springt bei t = 0 auf über 200, weil die Strömung aus der Ruhe schlagartig anliegt — den Maßstab der Achse bestimmte: von −7,3 bis 8,0, und alles Interessante lag als Strich dazwischen.

Behoben. Die Bahn des Netzwerkwegs wird mitgeschrieben; der Maßstab kommt aus der Zeit nach einem Zehntel des Laufs; Werte außerhalb werden abgeschnitten statt die Kurve auslaufen zu lassen. Die Probe sucht jetzt nach **den beiden Kurvenfarben selbst** — Blau- und Rotanteil getrennt gezählt.

Lehre: Ein Prüfposten muss das messen, was in seinem Namen steht. „Beide Wege" heißt: beide suchen, nicht Buntheit zählen. Es ist derselbe Fehlertyp wie in B17 — eine Prüfung, die zählt statt zu vergleichen, findet nichts Fehlendes.

B23 Die Seite ersetzte Marken im eigenen Datensatz

Auf Freds Einwand hin liegen seit Fassung 1.5 alle Programme im Wortlaut in der Seite — damit sich nachrechnen lässt, worauf das Prüfprotokoll verweist. Danach fiel die Knopfprobe von 62 auf 48 von 55, und der Browser meldete beim Laden `Invalid or unexpected token`.

Die Ursache liegt eine Ebene tiefer, als sie aussieht. `baue_labor.py` setzt die Teile der Seite über Marken ein: `@@KERN@@`, `@@LABOR@@`, `@@DATEN@@`. Das geschah **nacheinander** — jede Ersetzung lief über das Ergebnis der vorigen. Solange nur Manuskripte und Programmtext eingesetzt wurden, fiel das nicht auf. Seit aber `baue_labor.py` **selbst** zu den mitgelieferten Quelltexten gehört, steht sein Text im Datensatz — und darin stehen jene Marken. Der

nächste Durchgang ersetzte sie **mitten im JSON**, und damit war der ganze `LABORDATEN`-Block zerschlagen: keine Zeitdaten, keine Quelltexte.

Warum die vorhandene Prüfung es beinahe durchgelassen hätte. Die Seite sah fast normal aus. Alle neun Reiter gingen auf, die Manuskripte standen da, der Rechenteil funktionierte — nur der Zeitverlauf blieb leer und der Programmreiter auch. Ein Blick auf die Seite hätte „sieht gut aus" ergeben. Gefunden haben es die Prüfposten, die **auf Inhalt** prüfen: „der angezeigte Quelltext ist wirklich Quelltext — 0 Zeichen".

Behoben. Ein einziger Durchgang mit `re.sub` über alle Marken: Jede wird genau einmal angesehen, und was eingesetzt wird, wird nicht noch einmal durchsucht.

Lehre: Eine Ersetzung, die über ihr eigenes Ergebnis läuft, ist eine Falle, die erst zuschnappt, wenn der eingesetzte Inhalt zufällig wie eine Marke aussieht. Das kann Jahre dauern — und tritt genau dann ein, wenn man den Erzeuger in sein eigenes Erzeugnis legt. Bei Textmarken gilt: **ein Durchgang, keine Rückkopplung.**